

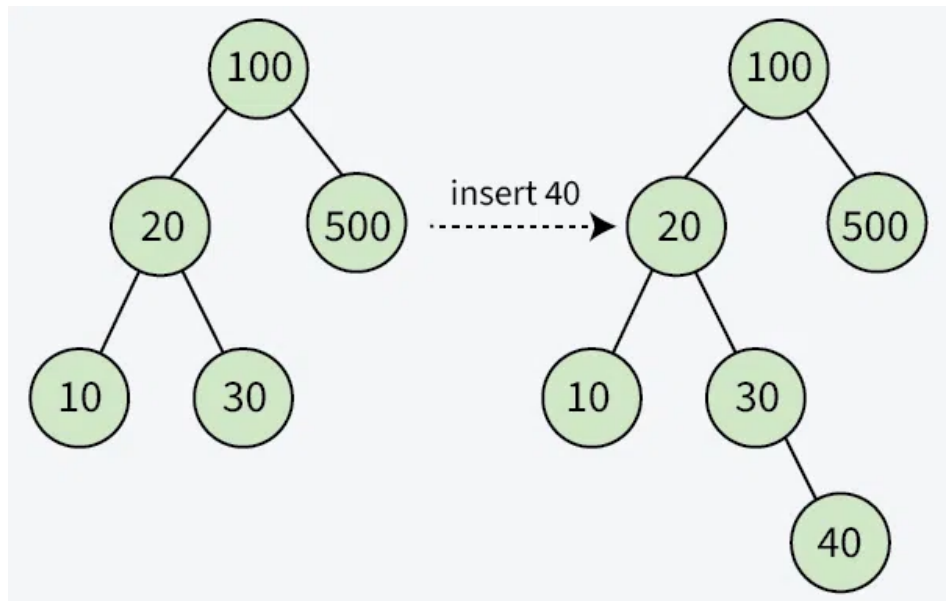
Insertion in Binary Search Tree (BST)

Last Updated : 16 May, 2025



Given a **BST**, the task is to insert a new node in this **BST**.

Example:

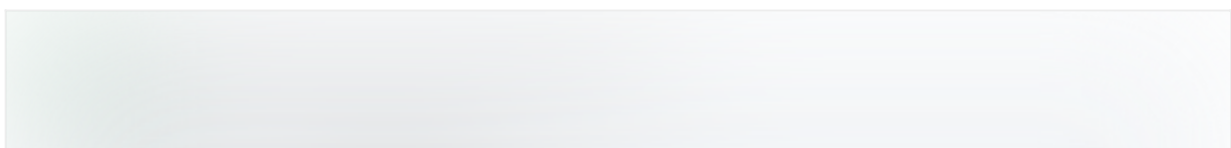


How to Insert a value in a Binary Search Tree:

A new key is always inserted at the leaf by maintaining the property of the binary search tree. We start searching for a key from the root until we hit a leaf node. Once a leaf node is found, the new node is added as a child of the leaf node. The below steps are followed while we try to insert a node into a binary search tree:

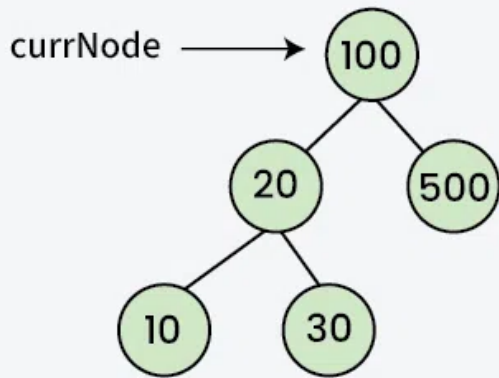
- Initialize the current node (say, **currNode** or **node**) with root node
- Compare the **key** with the current node.
- **Move left** if the **key** is less than or equal to the current node value.
- **Move right** if the **key** is greater than current node value.
- Repeat steps 2 and 3 until you reach a leaf node.
- Attach the **new key** as a left or right child based on the comparison with the leaf node's value.

Follow the below illustration for a better understanding:



01
Step

Consider the following BST

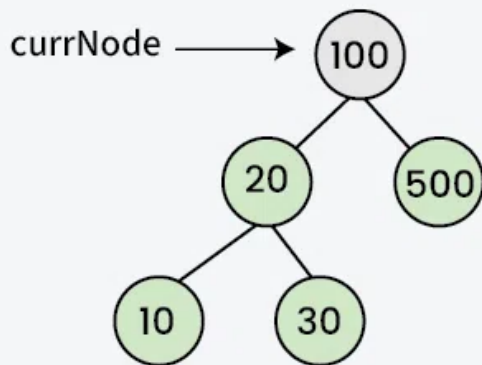


key = 40 (the node to be inserted)

Insertion in BST

02
Step

Comparing key with root node

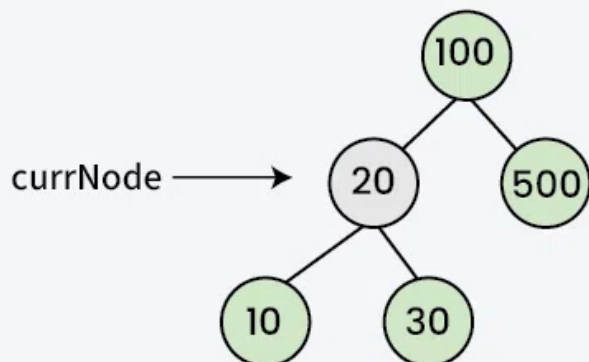


Since 100 is greater than key(40), move the pointer to the left child (20).

Insertion in BST

03
Step

Comparing key with left child root node

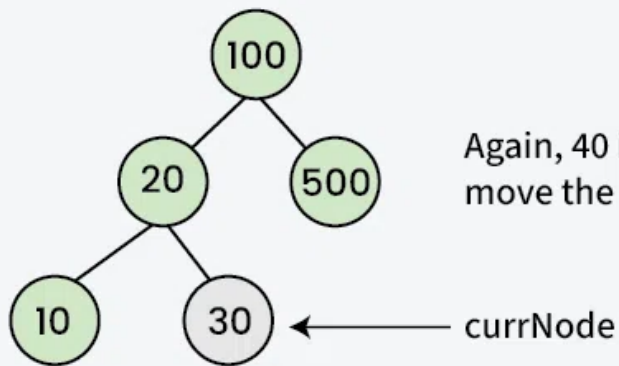


Since 20 is less than key(40), move the pointer to the right child (30).

Insertion in BST

04
Step

Comparing key with the right child of 20

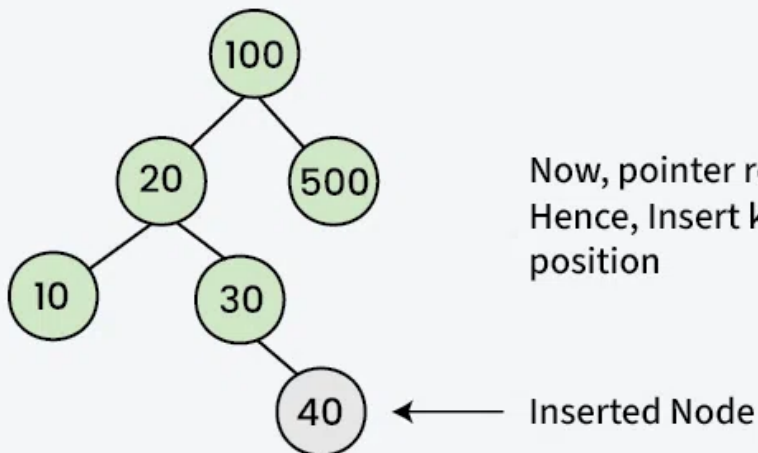


Again, 40 is greater than key(30),
move the pointer to the right of 30.

Insertion in BST

05
Step

Insert item to the right of 30



Now, pointer refers to null.
Hence, Insert key(40) at this
position

Insertion in BST

Insertion in Binary Search Tree using Recursion:

Below is the implementation of the insertion operation using recursion.

C++14

C

Java

Python

C#

JavaScript

```
#include <iostream>
using namespace std;

struct Node {
    int key;
    Node* left;
    Node* right;
    Node(int item) {
        key = item;
        left = NULL;
        right = NULL;
    }
};

// A utility function to insert a new node with
// the given key
Node* insert(Node* node, int key) {

    // If the tree is empty, return a new node
    if (node == NULL)
        return new Node(key);

    // If the key is already present in the tree,
```

```

// return the node
if (node->key == key)
    return node;

// Otherwise, recur down the tree/ If the key
// to be inserted is greater than the node's key,
// insert it in the right subtree
if (node->key < key)
    node->right = insert(node->right, key);

// If the key to be inserted is smaller than
// the node's key, insert it in the left subtree
else
    node->left = insert(node->left, key);

// Return the (unchanged) node pointer
return node;
}

// A utility function to do inorder tree traversal
void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->key << " ";
        inorder(root->right);
    }
}

// Driver program to test the above functions
int main() {
    // Creating the following BST
    //      50
    //     /  \
    //    30   70
    //   / \  / \
    //  20 40 60 80

    Node* root = new Node(50);
    root = insert(root, 30);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 70);

```

```

    root = insert(root, 60);
    root = insert(root, 80);

    // Print inorder traversal of the BST
    inorder(root);

    return 0;
}

```

Output

```
20 30 40 50 60 70 80
```

Time Complexity:

- The worst-case time complexity of insert operations is $O(h)$ where h is the height of the Binary Search Tree.
- In the worst case, we may have to travel from the root to the deepest leaf node. The height of a skewed tree may become n and the time complexity of insertion operation may become $O(n)$.

Auxiliary Space: The auxiliary space complexity of insertion into a binary search tree is $O(h)$, due to recursive stack.

Insertion in Binary Search Tree using Iterative approach:

Instead of using recursion, we can also implement the insertion operation iteratively using a **while loop**. Below is the implementation using a while loop.

C++
C
Java
Python
C#
JavaScript

```

#include <iostream>
using namespace std;

struct Node {
    int key;
    Node* left;
    Node* right;
    Node(int item)
    {

```

```

        key = item;
        left = NULL;
        right = NULL;
    }
};

Node* insert(Node* root, int x)
{
    Node* temp = new Node(x);

    // If tree is empty
    if (root == NULL)
        return temp;

    // Find the node who is going
    // to have the new node temp as
    // its child. The parent node is
    // mainly going to be a leaf node
    Node *parent = NULL, *curr = root;
    while (curr != NULL) {
        parent = curr;
        if (curr->key > x)
            curr = curr->left;
        else if (curr->key < x)
            curr = curr->right;
        else
            return root;
    }

    // If x is smaller, make it
    // left child, else right child
    if (parent->key > x)
        parent->left = temp;
    else
        parent->right = temp;
    return root;
}

// A utility function to do inorder
// tree traversal
void inorder(Node* root)

```

```

{
    if (root != NULL) {
        inorder(root->left);
        cout << root->key << " ";
        inorder(root->right);
    }
}

// Driver program
int main()
{
    // Creating the following BST
    //      50
    //     /  \
    //    30   70
    //   / \  / \
    //  20 40 60 80

    Node* root = new Node(50);
    root = insert(root, 30);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 70);
    root = insert(root, 60);
    root = insert(root, 80);

    // Print inorder traversal of the BST
    inorder(root);

    return 0;
}

```

Output

```
20 30 40 50 60 70 80
```

The **time complexity** of **inorder traversal** is **$O(h)$** , where h is the height of the tree.
 The **Auxiliary space** is **$O(1)$**

Related Links:

- [Binary Search Tree Search Operation](#)
- [Binary Search Tree Delete Operation](#)

 Comment

More info 

Advertise with us

Next Article 

Searching in Binary Search Tree (BST)

Similar Reads

Binary Search Tree

A Binary Search Tree (or BST) is a data structure used in computer science for organizing and storing data in a sorted manner. Each node in a Binary Search Tree has at most two children, a left child and a right...

 3 min read

Introduction to Binary Search Tree

Binary Search Tree is a data structure used in computer science for organizing and storing data in a sorted manner. Binary search tree follows all properties of binary tree and for every nodes, its left subtree...

 3 min read

Applications of BST

Binary Search Tree (BST) is a data structure that is commonly used to implement efficient searching, insertion, and deletion operations along with maintaining sorted sequence of data. Please remember th...

 3 min read

Applications, Advantages and Disadvantages of Binary Search Tree

A Binary Search Tree (BST) is a data structure used to storing data in a sorted manner. Each node in a Binary Search Tree has at most two children, a left child and a right child, with the left child containing...

 2 min read

Insertion in Binary Search Tree (BST)

Given a BST, the task is to insert a new node in this BST.Example: How to Insert a value in a Binary Search Tree:A new key is always inserted at the leaf by maintaining the property of the binary search...

 15 min read

Searching in Binary Search Tree (BST)

Given a BST, the task is to search a node in this BST. For searching a value in BST, consider it as a sorted array. Now we can easily perform search operation in BST using Binary Search Algorithm. Input: Root of...

🕒 7 min read

Deletion in Binary Search Tree (BST)

Given a BST, the task is to delete a node in this BST, which can be broken down into 3 scenarios: Case 1. Delete a Leaf Node in BST Case 2. Delete a Node with Single Child in BST Case 3. Delete a Node with Two Children in BST

🕒 10 min read

Binary Search Tree (BST) Traversals – Inorder, Preorder, Post Order

Given a Binary Search Tree, The task is to print the elements in inorder, preorder, and postorder traversal of the Binary Search Tree. Input: A Binary Search Tree Output: Inorder Traversal: 10 20 30 100 150...

🕒 10 min read

Balance a Binary Search Tree

Given a BST (Binary Search Tree) that may be unbalanced, the task is to convert it into a balanced BST that has the minimum possible height. Examples: Input: Output: Explanation: The above unbalanced BST...

🕒 10 min read

Self-Balancing Binary Search Trees

Self-Balancing Binary Search Trees are height-balanced binary search trees that automatically keep the height as small as possible when insertion and deletion operations are performed on the tree. The height...

🕒 4 min read
